Chapter 4

RESULTS AND DISCUSSION

This chapter presents the analysis, presentation, discussion, and findings of the study gathered by the researchers. Data collected from the six (6) Program Chairpersons of the College of Engineering and Information Technology (CEIT), selected IT experts, and students who experienced prerequisite violations are systematically organized, interpreted, and analyzed. The presentation of results corresponds directly to the Statement of the Problem: (1) the current process for evaluating students' prerequisite compliance with the curriculum, (2) the problems encountered in that process, (3) the design of a curriculum audit system to ensure prerequisite compliance, and (4) the level of compliance of the proposed system, Auditrix, to the ISO/IEC 25010 software quality standard. The results of the testing done on the developed system are presented before SOP No. 4.

Process of the Existing System (SOP No. 1)

Based on data gathered through preliminary structured interviews with the six (6) Program Chairpersons at KSU-CEIT, the current mechanism for evaluating student prerequisite

compliance remains predominantly dependent on decentralized, manual methods. Kalinga State University maintains a central Management Information System (MIS) that stores historical final grades and manages registration data; however, this MIS was not built to analyze whether a student's completed subjects actually follow the prescribed curriculum sequence. As a result, prerequisite checking still relies on traditional paper-based or legacy cross-referencing even with the MIS in place, and inconsistencies are frequently not flagged during the enrollment phase itself.

During the compressed enrollment period, the manual curriculum audit process operates as follows:

Digital Subject Selection. Students access the MIS portal to initiate enrollment and select the subjects they wish to add. Many students use the "Enroll All" button, which automatically queues every available subject into a pending status without validating historical grades against prerequisite or corequisite requirements. Because students can view their own records but are not proactively warned about ineligibility, they may select subjects they are not yet qualified to take.

Manual Record Auditing. The Program Chairperson must then manually open each pending enrollment request, going through the student's record line-by-line and cross-referencing completed courses and passing marks (≤3.00) against the physical curriculum map or program checklist.

Human Error and Validation. The Chairperson attempts to filter out unauthorized subjects before finalizing the enrollment data. Because this process is repetitive and constrained by a compressed timeline, human error is unavoidable, allowing prerequisite violations to go undetected until the final graduation audit — a bottleneck that forces affected students into additional semesters to satisfy missed requirements.

[INPUT]	[PROCESS]	[OUTPUT]
- Student Historical Grades on MIS Portal.	- Automated queuing of unverified subjects via "Enroll all" option	- Approved or Finalized Enrollment Ledger.
- Official KSU-CEIT Curriculum map.	- Line-by-line manual audit of pending registrations against passing records	- Undetected Prerequisite Violations in the System
- Pending Unverified Subject Enrollee logs.	- Human error filtration slips during manual check	- Late-discovered violations surfacing at graduation audit

Figure 4.1. Input-Process-Output Diagram of the Existing System

Problems Encountered in the Current System (SOP No. 2)

The structural reliance on manual human review within a compressed timeline creates operational vulnerabilities. Data drawn from the assessment tools highlight three critical administrative issues:

1. Late-Detection Blindspots. Because the current MIS does not enforce structural blocks during digital enrollment selection, prerequisite violations frequently go unnoticed. These discrepancies are often discovered years later during the final graduation audit, forcing senior students into unexpected residency extensions and heavy financial strain.

2. Absence of Automated Compliance-Checking Tools. While the MIS reliably stores student grades and registration

records, it has no built-in logic to cross-reference a student's completed subjects against the official curriculum structure. This places the entire burden of compliance verification on the Chairperson's manual review rather than on the system itself, leaving no automated safeguard between data entry and eligibility determination.

3. Limited Student Awareness of Prerequisite Requirements. Although students can view their own academic records through the MIS, the system does not proactively alert them when they are ineligible for a subject. Students may therefore unknowingly attempt to enroll in courses without having completed the necessary prerequisites, adding to the volume of cases the Chairperson must catch downstream.

Figure 4.2. Ishikawa (Fishbone) Diagram of the Problems in the Existing System

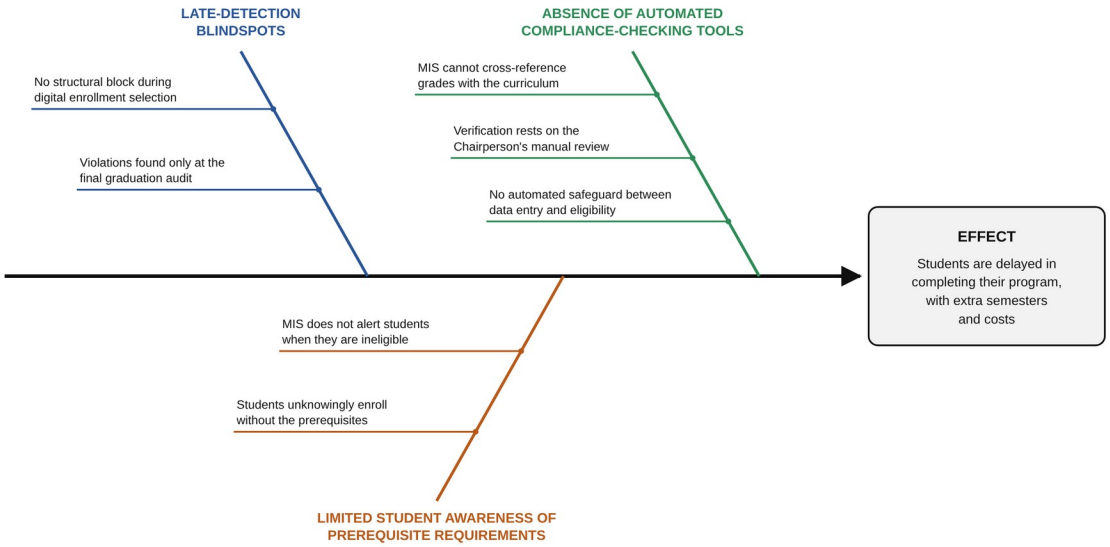


Figure 4.2 presents these three problems in an Ishikawa (fishbone) diagram, with each problem as a branch and its causes listed under it: Late-Detection Blindspots (no structural block during digital enrollment selection; violations found only at the final graduation audit), Absence of Automated Compliance-Checking Tools (the MIS cannot cross-reference grades with the curriculum; verification rests on the Chairperson's manual review; no automated safeguard between data entry and eligibility), and Limited Student Awareness of Prerequisite Requirements (the MIS does not alert students when they are ineligible; students unknowingly enroll without the prerequisites). Together, they lead to the same effect: students who are delayed in completing their program, with extra semesters and costs.

Narrative Description of the Proposed System (SOP No. 3)

To address the gaps identified above, the researchers designed and developed Auditrix, a web-based curriculum audit system for Kalinga State University that automatically checks every subject a student has taken against the prerequisites, co-requisites, and year-standing requirements of the student's curriculum. It was built for any college and program, and was tested with the BSCpE, BSIT, and BSCE

programs of the CEIT. The system was built using the Iterative and Incremental Development (IID) model, allowing the Audit Engine's logic to be reviewed early and refined across three functional increments: (1) curriculum, student and grade records with a basic audit; (2) curriculum versions, imports, student movements and the redesigned interface; and (3) the term-based audit, automatic year levels, and the regular and irregular classification.

The system runs on Next.js 16 (App Router and server actions) with React 19 and TypeScript, styled with Tailwind CSS 4 in a KSU green theme designed for responsiveness across desktop, tablet, and mobile. Supabase serves as the backend-as-a-service, hosting the PostgreSQL database, authentication, and Row-Level Security (RLS) policies; the database schema is built step by step by 30 migration scripts. Curricula are exported to PDF with @react-pdf/renderer. Interface flows were prototyped as high-fidelity mockups in Figma prior to implementation.

At the core of Auditrix is a TypeScript-based audit engine that checks every subject of the student's curriculum against the student's grade records and gives each subject one

of five statuses, shown in Table 4.1 together with the subject's grade and the term in which it was taken:

Status	Meaning
Completed	Passed, and the pass counts because the subject was taken in order.
Available	Can be taken now because its requirements are met, or must be retaken because it was failed or taken out of order.
Pending	In progress, INC, or waiting for another subject's final grade.
Unavailable	Requirements not met yet; the missing ones are listed (e.g., "Needs: CC 104").
Violation	Taken before a requirement was met; the reason is given (e.g., "CC 101 was taken in the same term (24-1)").

Table 4.1. Status of a Subject in the Student Audit

The statuses are not stored at the moment a grade is entered. Instead, every time a student's audit is shown, the engine compares the term in which each subject was taken with the term in which each of its requirements was passed. Because of this, grades can be encoded in any order and the result stays the same, and a violation remains on the student's

record until the subject is retaken after its requirements are met. This design replaced the system's first checker, which stored a permanent flag at the moment of entry; the reasons for the change are discussed under Testing of the System.

The engine evaluates four requirement types — prerequisite, co-requisite, year standing, and "all subjects completed" — closing the exact gap identified in SOP 1 and 2, where the existing MIS had no equivalent logic. The rules it follows, and their sources, are described below.

Terms. Terms are written as on KSU records: the first two digits of the school year and the semester (1 for the first semester, 2 for the second, and S for the midyear). A midyear carries the year of the semester after it, so terms are ordered 25-1, 25-2, 26-S (midyear), then 26-1.

Prerequisites. The KSU Student Handbook requires a prerequisite to be passed prior to taking a subject and nullifies the credit of a subject taken without it. Accordingly, a prerequisite must be passed in a term before the term in which the subject was taken; passing it in the same term is a violation. Taking a subject while its prerequisite is still INC is also a violation, since an INC counts as passed only from the term in which it was resolved.

A subject taken out of order earns no credit: it does not satisfy later prerequisites, does not count toward units or year level, and has to be retaken. If a prerequisite's grade from an earlier term is still in progress, the subject is Pending rather than a violation.

Co-requisites. The Student Handbook defines co-requisites as subjects taken simultaneously, in the same semester. In Auditrix, they pair both ways, even when only one of the two subjects lists the other. Both must be taken in the same term; taking only one of them, or taking them in different terms, is a violation. If one is failed, neither counts and both are retaken.

Year level. Following the KSU Student Handbook (2020 edition) and the KSU Operations Manual, which specifies that a student's year level should follow what the student has finished in the curriculum, the year level is computed from the share of the curriculum's units passed, as shown in Table 4.2. Passing every subject of the earlier years also moves a student up, even if the units fall slightly short. Only passes that count, and only those earned before the current semester, are included, so students move up when the Dean moves the college to a new semester. The year level given at

registration or import is used as a starting point until the grades show more, and a chairperson can set a year level by hand for a special case and set it back to automatic.

Year level	Share of the curriculum's units passed
1st year (Freshman)	Less than 25%
2nd year (Sophomore)	25% to 50%
3rd year (Junior)	More than 50% (exactly 50% stays 2nd year)
4th year (Senior)	75% or more

Table 4.2. Year Level by Share of Curriculum Units Passed

Year standing. A year-standing requirement (e.g., "Third year standing") is met by being on that year level. For a subject already taken, the engine uses the student's year level going into that term, counting only what was passed before it. When nothing was recorded for that term, and the grades alone do not show the standing, for example, because older grades have not been encoded yet, the subject is marked "Year standing: please verify" (in amber) instead of being flagged as a violation.

Regular and irregular students. Per the Student Handbook and the Operations Manual, a student is regular when enrolled in the full prescribed load for the current semester, and irregular when enrolled in less; the missing subjects are

listed. The prescribed load is taken to be the curriculum's subjects for the student's year level and the current semester, not counting subjects already passed in an earlier term, and back subjects taken on top of a full load do not make a student irregular. Enrollment is known from the grades (or "in progress" records) of the current semester; with nothing recorded, the status is "Not determined," and the student is not counted either way. At the start of a semester, the chairperson marks the subjects each student is taking as In progress, so the status is known before any grades come in.

Grade entry. Grades are entered per term for the subjects of a chosen curriculum year and semester, starting on the current semester. Only valid KSU grades are accepted: 1.0, 1.25, 1.5, 1.75, 2.0, 2.25, 2.5, 2.75, and 3.0 (passing), 5.0 (failing), and INC, and a subject the student is taking this semester can be marked In progress. Several subjects can be ticked and marked In progress at once; ticking all leaves out the subjects the student cannot take yet (e.g., "Can't take yet. Needs: MATH 111"), which can still be ticked one by one if the student is enrolled in them anyway. Only changed grades are saved, and changing an existing grade is treated as a correction that asks for confirmation. A final grade for an

INC keeps the subject in the term it was taken and records the term in which the INC was resolved (shown as "INC → 25-1"). Since only the latest attempt of a subject is kept as the student's record, a grade from an earlier term than the recorded one is refused unless the chairperson confirms that it corrects a wrongly entered term. After saving, a message names any subject that has just become a violation, and every grade ever entered is kept in the student's grade logbook together with who entered it and when. The database writes this logbook itself, so its entries cannot be edited or deleted, and it accepts only KSU grades that match their status and well-formed terms.

Curriculum management. Each program keeps curriculum versions by effective year (e.g., BSCpE 2019 and 2023), and a new version can copy an existing one. Each subject has a code, title, units, year level and semester (including midyear), and its requirements. Curricula can be imported from an Excel (.xlsx or .xls) or CSV file, in a format designed by the researchers, with a preview of what will be added, changed or removed before saving, exported to CSV or PDF, and every change is kept in a per-version change history showing who changed what, and when.

Students and student movements. Students are registered one at a time or imported from an Excel (.xlsx or .xls) or CSV file, either the system's template or a class list exported from the KSU MIS (its "Export to Excel" file, in the format the Program Chairpersons provided), and ID numbers are checked against the KSU format (e.g., 24-113792). When a student shifts, the releasing chairperson files a shift request and the receiving program's chairperson accepts the student into one of their curriculum versions. Transfers in and out and drops are recorded as well, and every movement is kept in the student's history. Marking a student as dropped or transferred out locks the record: the database refuses changes to the student's grades, details and year level until the chairperson reopens the record, which is also kept in the history. A Student Movements page lists every shift, transfer, drop and reopening, with search and filters. The student list can be searched from the top bar of any page and filtered by year level, curriculum version and status, and the reason for each student's status is shown under the name (e.g., "Out of order: PHYS 201").

Dashboard. A "Needs attention" banner at the top lists what to act on, such as students with violations, students

with nothing recorded for the current semester, or an unset semester, each linked to the matching filtered student list. Per program, the dashboard shows the total number of students, the regular and irregular students (and how many are not yet determined), the students with violations, the curriculum versions, and the number of shifts, transfers and drops. Four charts are built only from recorded data: regular, irregular and not determined students; students by year level from Freshman to Senior; bottleneck subjects, where the most students' latest grade is 5.0 or INC; and the subjects most often taken out of order. Each chart has a short explanation, shows the exact numbers on hover, and can be viewed as a table. The Dean sets the current semester from the dashboard and compares the programs of the college side by side, but does not open individual students' records, which remain the chairperson's work.

Access control and data scoping. Supabase RLS scopes visibility by role: Program Chairpersons see only the data of their own program, while Deans see their whole college. Operations that must all succeed together, such as creating a program with its chairperson account, replacing a chairperson, or accepting a shifting student, run as single

database transactions. A new chairperson receives a temporary password from the Dean and must choose their own at first login, guided by a checklist that ticks each password rule as it is met, and actions that are hard to undo, such as dropping a student or saving grades, ask for confirmation first. Every page checks the user's role, so a page meant for another role cannot be opened by typing its address; users are signed out after 30 minutes without activity; and an account is blocked for 15 minutes after five wrong passwords.

Features of the Proposed System

- Role-based login for Deans and Chairpersons, with a required password change at first login
- Term-based Audit Engine checking prerequisites, co-requisites, year standing and "all subjects completed"
- Student registration and import from Excel or CSV (the template or the KSU-MIS "Export to Excel" file)
- Student list with search from any page, filters and the reason for each student's status, and an audit page showing each subject's status, grade, term and the reason for any violation
- Term-based grade entry with KSU grade values, In progress marking for several subjects at once, INC

completion, earlier-term protection and a database-written grade logbook

- Curriculum management with versioning (e.g., BSCpE 2019 vs. BSCpE 2023), change history, and import with preview
- Shift requests and shift-ins, transfers and drops, with a per-student history, locked records for dropped and transferred-out students, and a Student Movements page
- Automatic year levels with chairperson override, and regular/irregular classification
- Dashboard per program with a Needs attention banner and charts, a side-by-side program comparison for the Dean, and the current semester set by the Dean
- Export of curriculum versions to CSV or PDF
- Security: role checks on every page, row-level security, a database-enforced audit trail, sign-out after 30 minutes of inactivity, and blocking after repeated wrong passwords

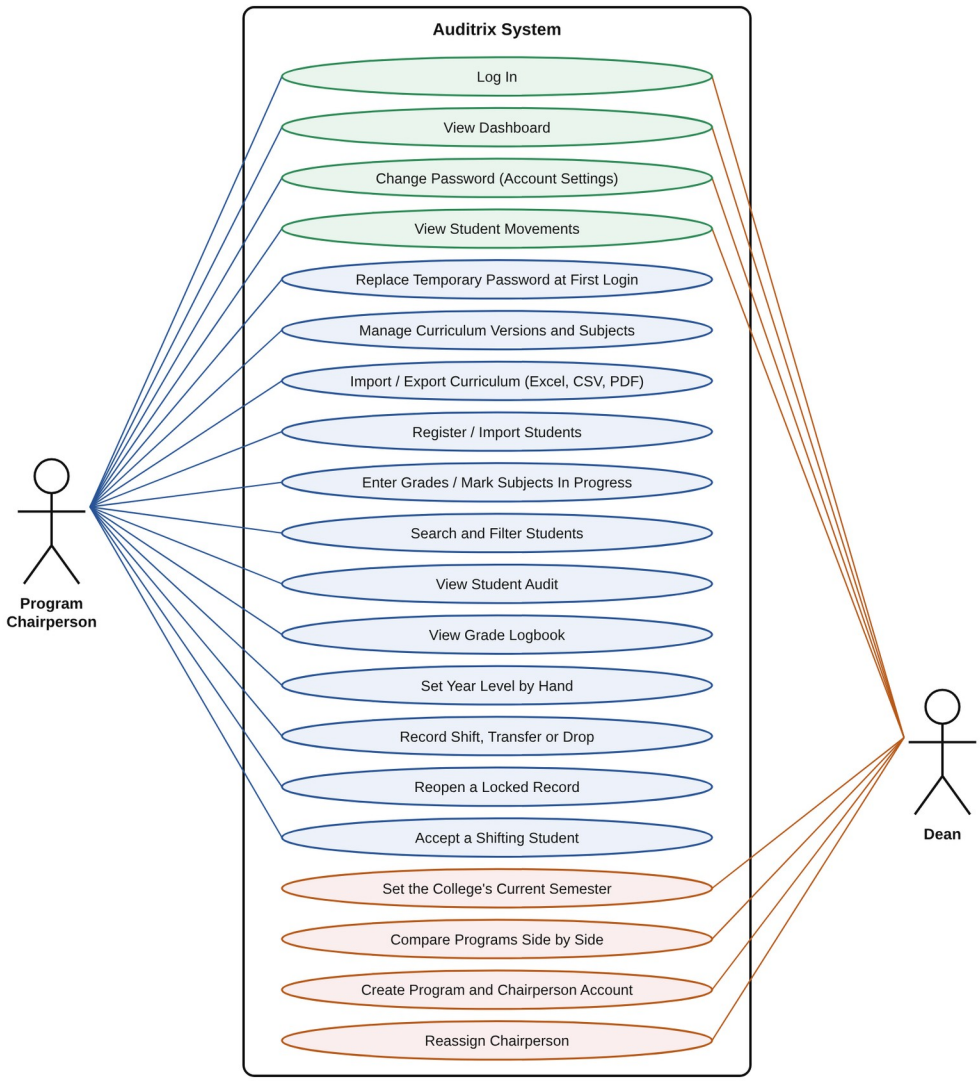
Scope boundaries. The system does not include subsystems for the Registrar, Accounting, or Library, and it is not connected to the KSU MIS. Its logic is limited to descriptive auditing (identifying compliance with existing rules); it does not perform predictive analytics, graduation monitoring, or AI-driven enrollment counseling.

Input-Process-Output (IPO) Diagram of the Proposed System

[INPUT]	[PROCESS]	[OUTPUT]
- Student lists (registered, or imported from Excel/CSV or the KSU-MIS export)	- Term-based audit of every subject against its requirements	- Status per subject (Completed / Available / Pending / Unavailable / Violation) with reasons
- Versioned curriculum: subjects with their prerequisites, co-requisites and year-standing requirements	- Year level from units passed; regular / irregular from the prescribed load	- Violations flagged as soon as grades are saved; subjects available for the next enrollment
- Grades per term (including INC and its resolution), the current semester, and shifts, transfers and drops	- Role-scoped review via RLS (Chairperson by program, Dean by college)	- Dashboard counts and charts, grade logbook, student movements, and curriculum CSV / PDF exports

Figure 4.3. Input-Process-Output Diagram of the Proposed System

Figure 4.4. Use Case Diagram of Auditrix



Green: used by both roles. Blue: Program Chairperson only. Orange: Dean only.
The audit itself runs automatically whenever a student's audit or the dashboard is shown.

Figure 4.4 shows the two actors of the system. The Program Chairperson interacts with most modules: managing curriculum versions (including their import and export), registering and importing students, entering grades, viewing each student's audit and grade logbook, setting a year level by hand, recording shifts, transfers and drops, and reopening

a locked record. The Dean oversees the college: viewing the dashboard of every program and comparing the programs, setting the current semester, creating programs with their chairperson accounts, and reassigning chairpersons. Both log in, view a dashboard and the Student Movements page, and change their password, and a new chairperson must replace the temporary password at first login. The audit itself needs no separate action: the system performs it whenever a student's audit or the dashboard is shown.

Figure 4.5. Activity Diagram of the Audit of a Subject

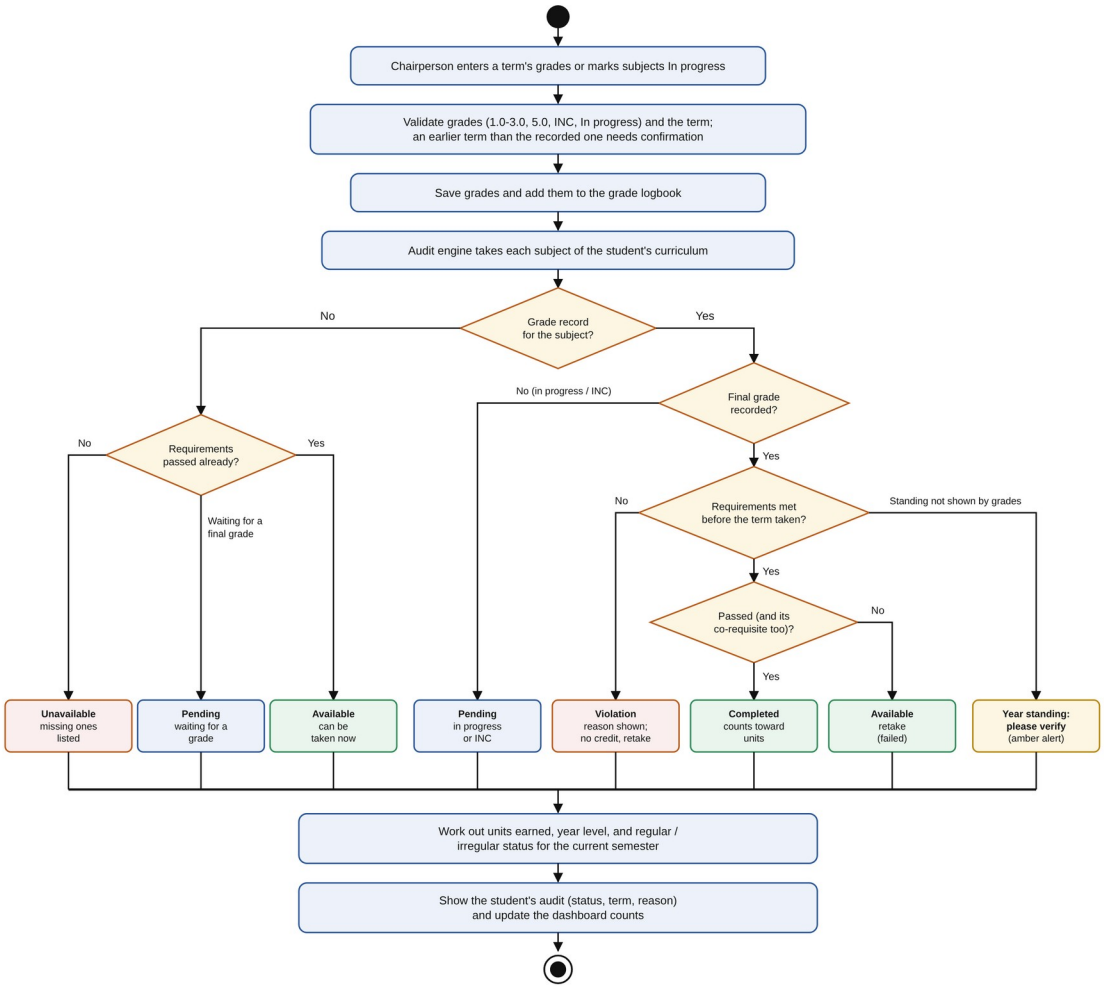
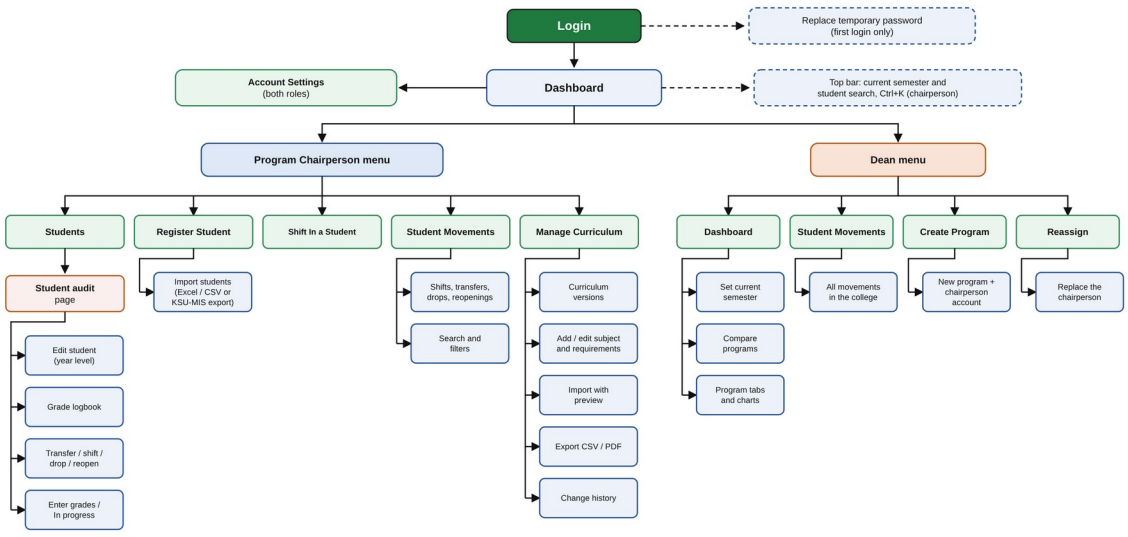


Figure 4.5 traces the audit of a single subject. After the chairperson saves a term's grades or marks subjects In progress, which are first checked for valid KSU grade values and for conflicts with a later recorded term, the audit engine looks at the student's record for each subject. A subject with no record is Available if its requirements have been passed, Pending if it is waiting for a requirement's final grade, and Unavailable otherwise, with the missing requirements listed.

A subject that is in progress or INC is Pending. For a graded subject, the engine checks whether every requirement was met before the term it was taken: a prerequisite passed in an earlier term, the co-requisite taken in the same term, and the year standing at that time. If not, the subject is a Violation with the reason given, earns no credit, and must be retaken; if the year standing cannot be confirmed from the grades, it is marked "Year standing: please verify". If the requirements were met, a passed subject whose co-requisite was also passed is Completed and counts toward units and year level, while a failed one is Available for retaking. The results then update the student's year level, regular or irregular status, and the dashboard.

Figure 4.6. Menu Diagram of Auditrix



Menu depth: Login -> Dashboard -> role menu -> page -> sub-page. Chairpersons reach a student's pages from Students or the top-bar search.

Figure 4.6 outlines the system's navigation. From Login (and the password change required at a chairperson's first login), users land on the Dashboard. A Program Chairperson's menu leads to Students, Register Student, Shift In a Student, Student Movements and Manage Curriculum, and a student search is available from the top bar of every page; from Students, each student's audit page opens the Edit Student, Grade Logbook, Transfer / Shift, and Enter Grades pages. A Dean's menu leads to the Dashboard, where the current semester is set and the programs are compared, Student Movements, Create Program and Reassign Chairperson. Account Settings is available to both roles.

Figure 4.7. User Interface of Auditrix

Figure 4.7 shows the main screens of the developed system, captured from the running application with sample CEIT data.

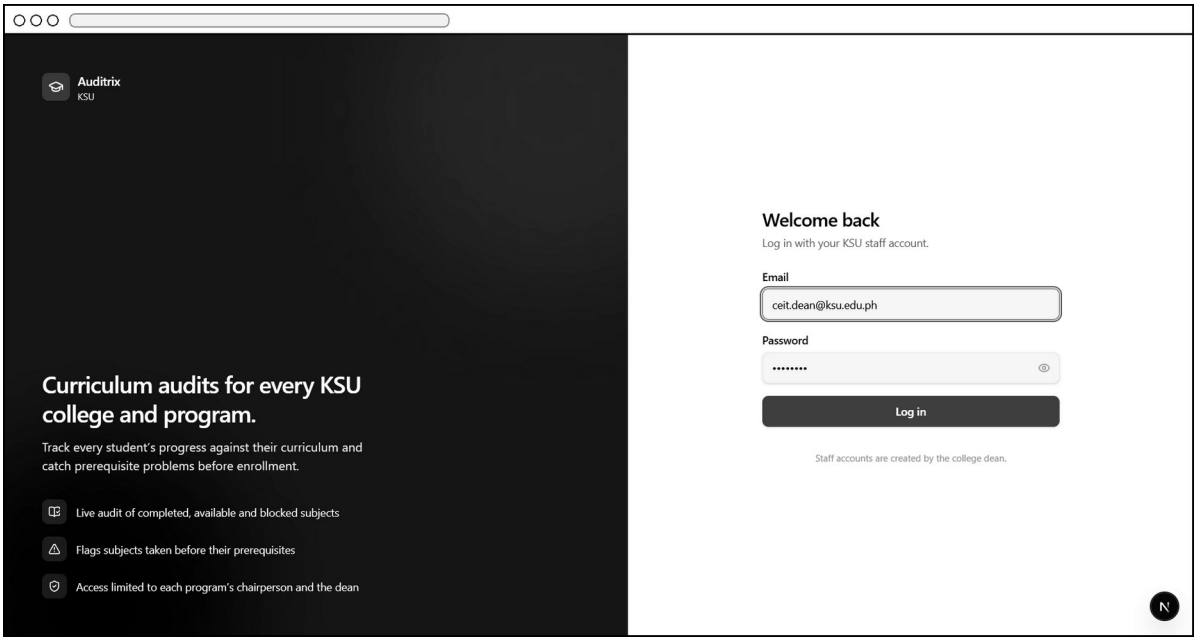


Figure 4.7a. Log In – Dean role

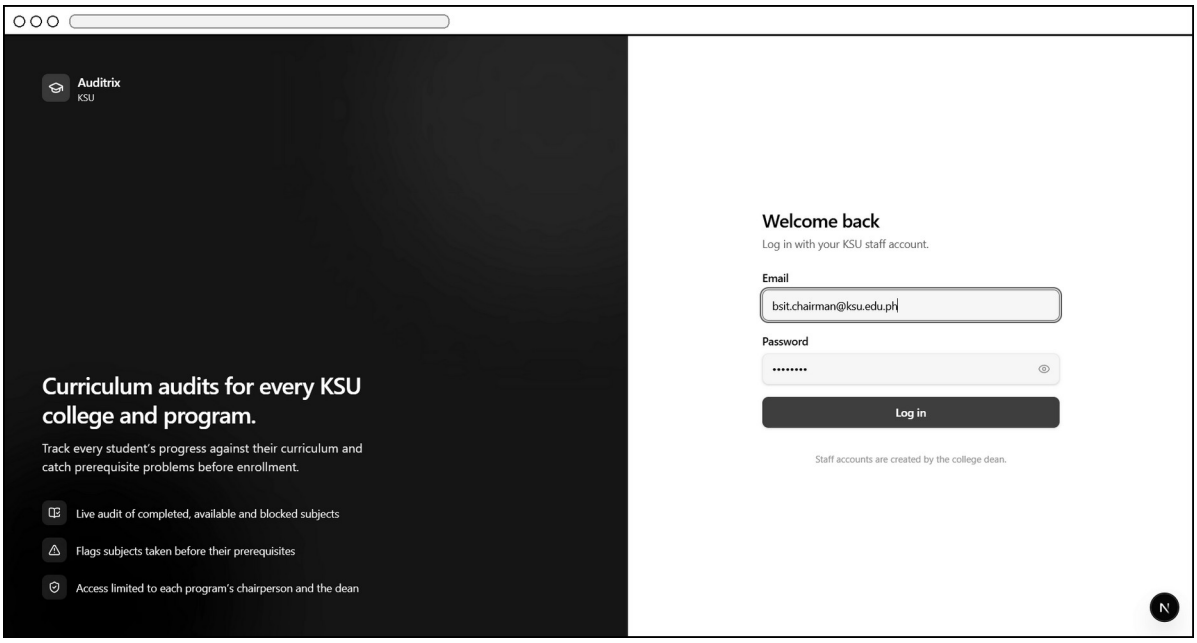


Figure 4.7b. Log In – Program Chairperson role

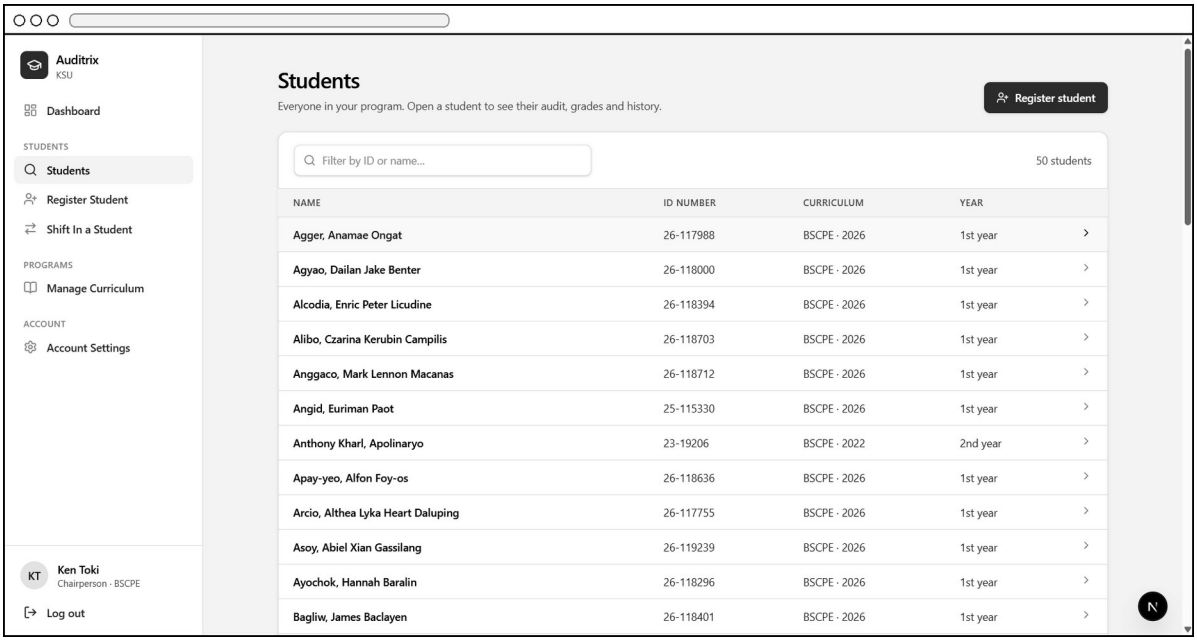


Figure 4.7c. Students – search and list view

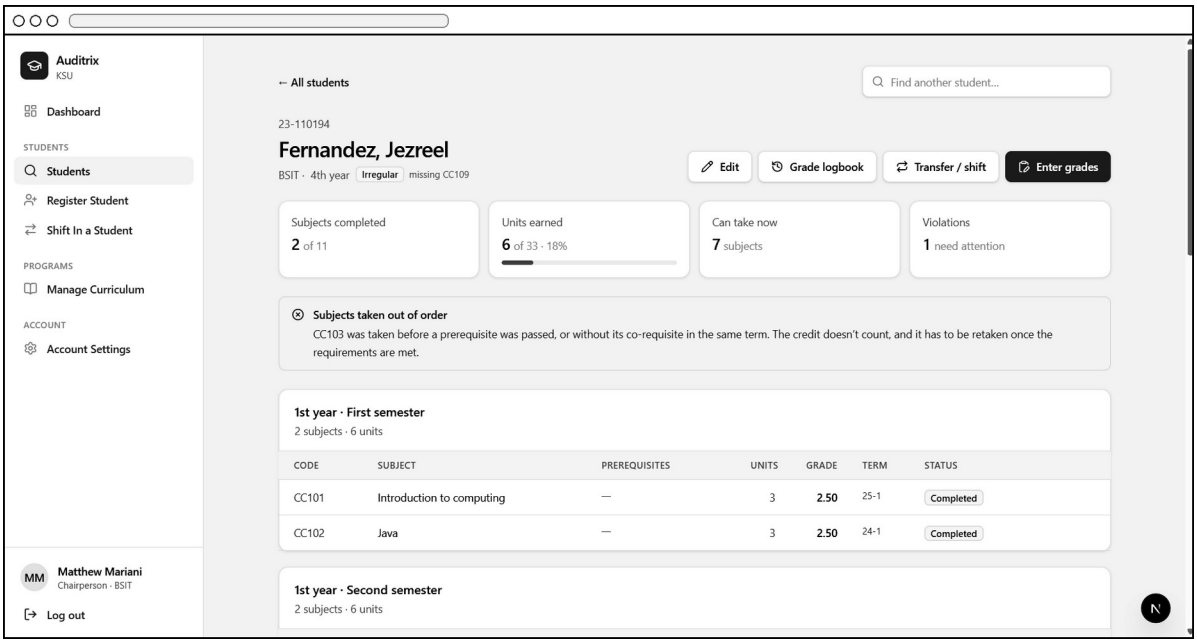


Figure 4.7d. Student Audit – subject taken out of order and Irregular status

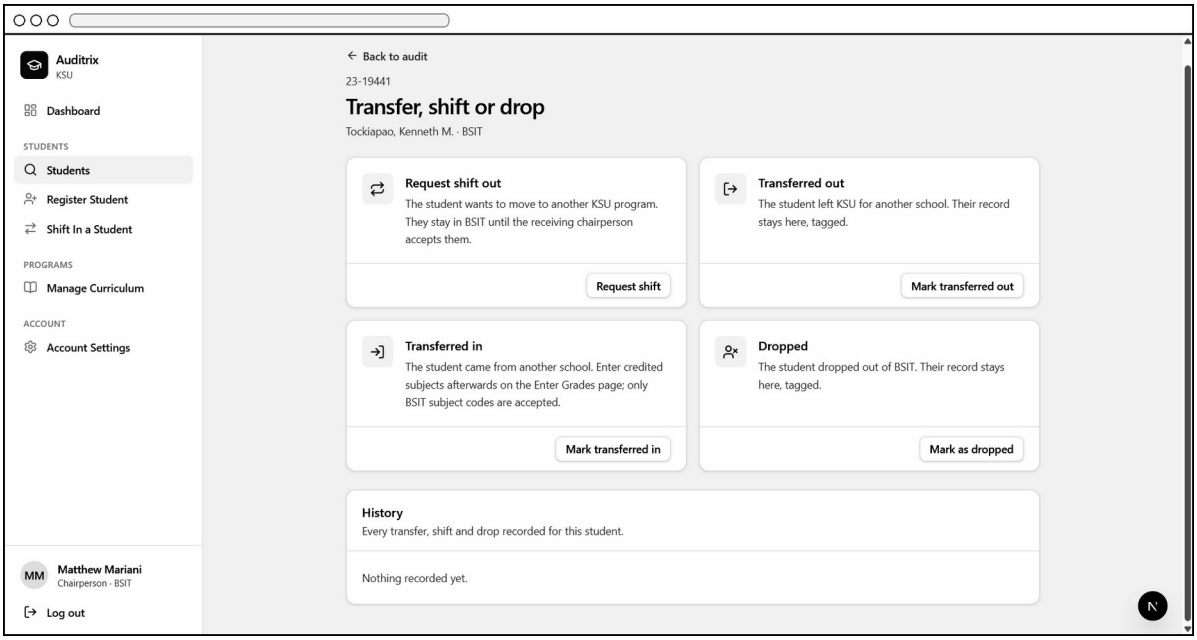


Figure 4.7e. Transfer / Shift Records

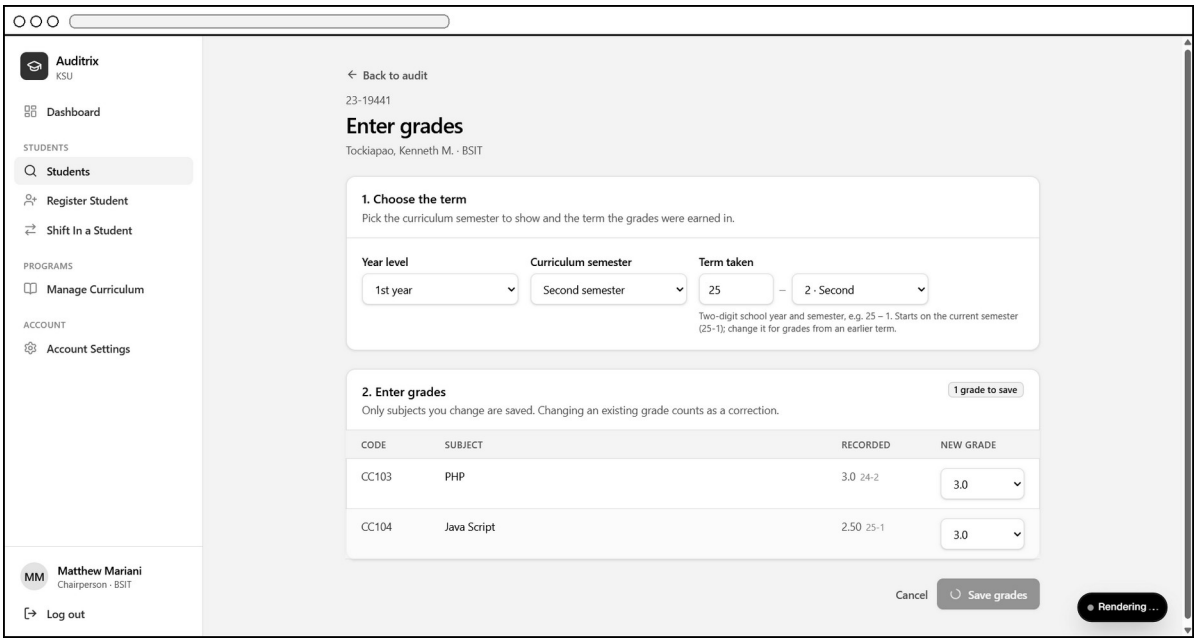


Figure 4.7f. Enter Grades – choosing the term

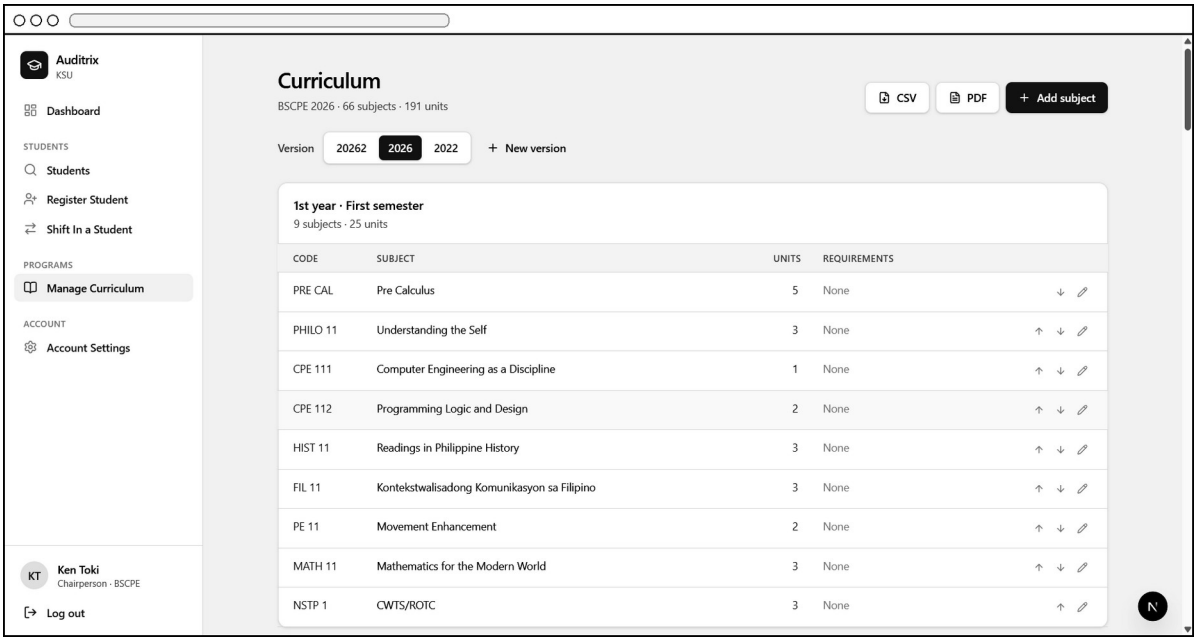


Figure 4.7g. Manage Curriculum Versions

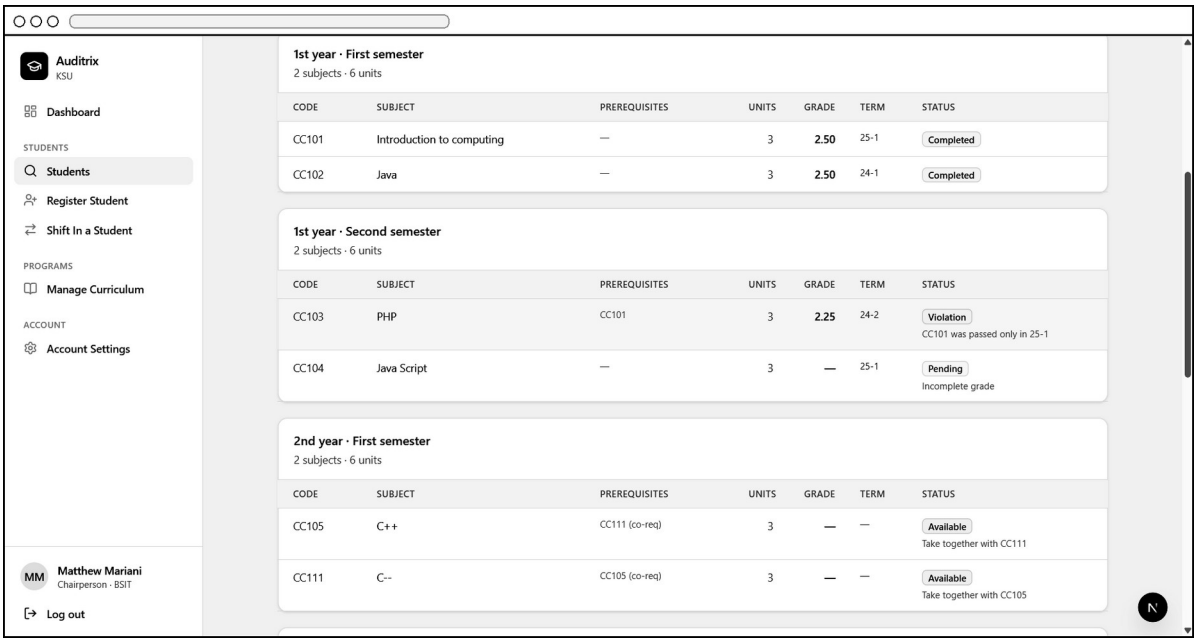


Figure 4.7h. Student Audit – status, term and reason for each subject

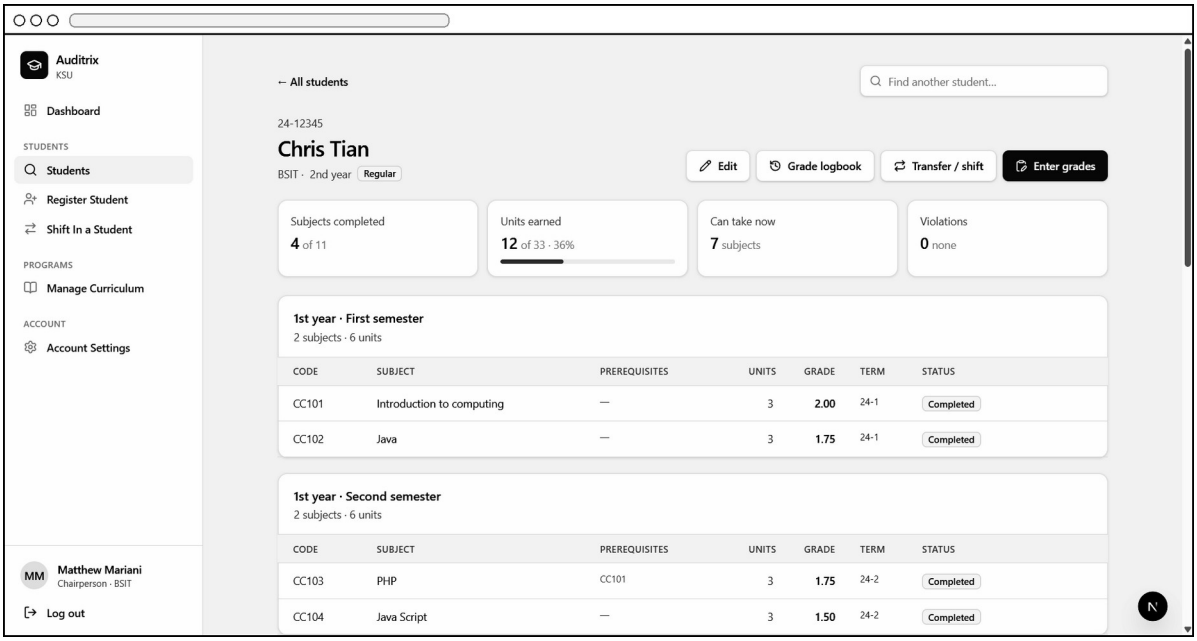


Figure 4.7i. Student Audit – fully compliant (Regular) example

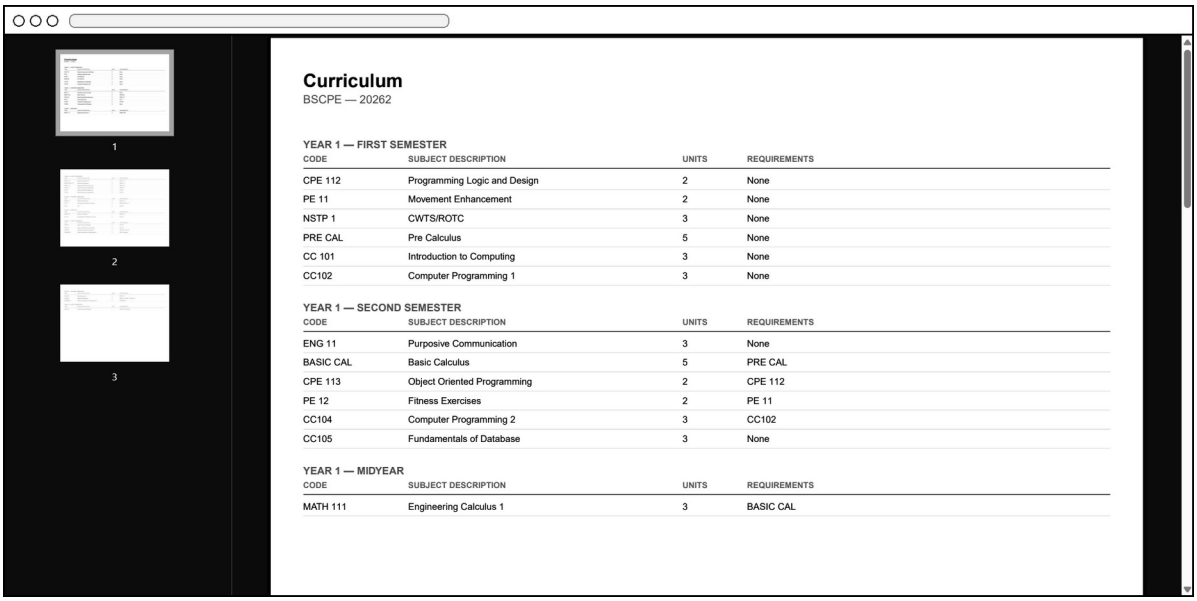


Figure 4.7j. Export Curriculum – PDF output

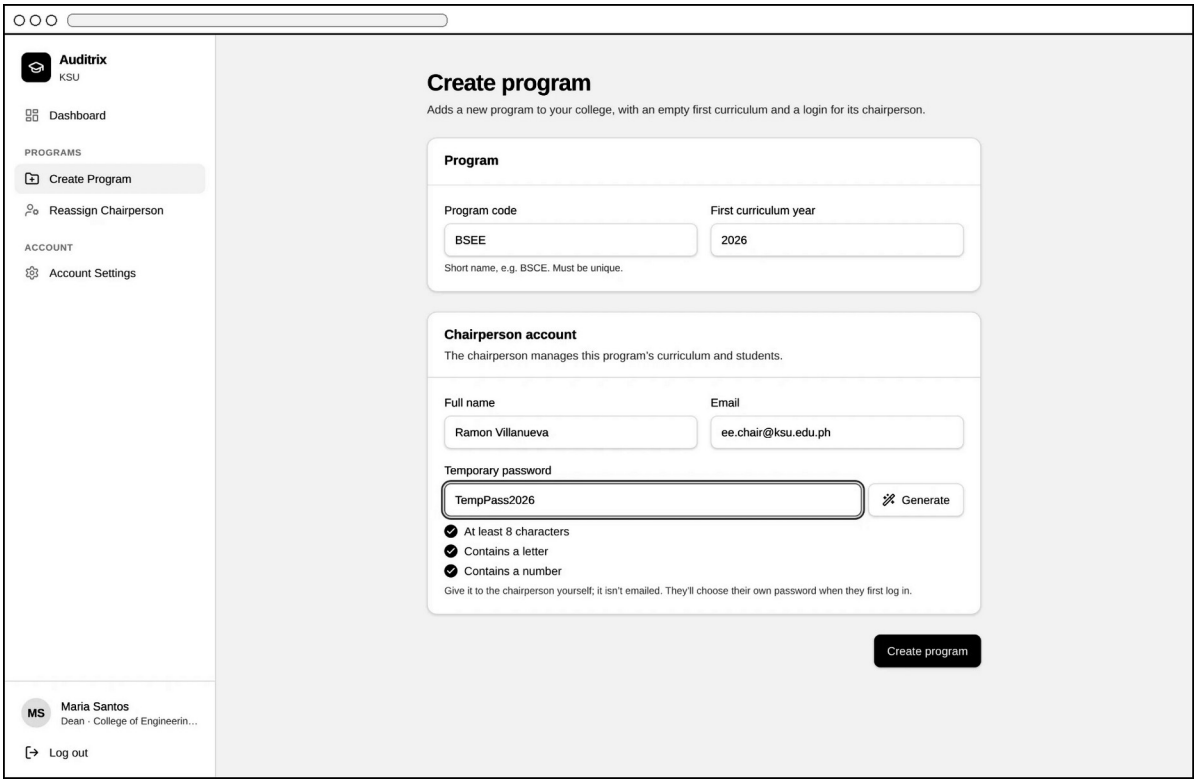


Figure 4.7k. Create Program — new chairperson account with a temporary password

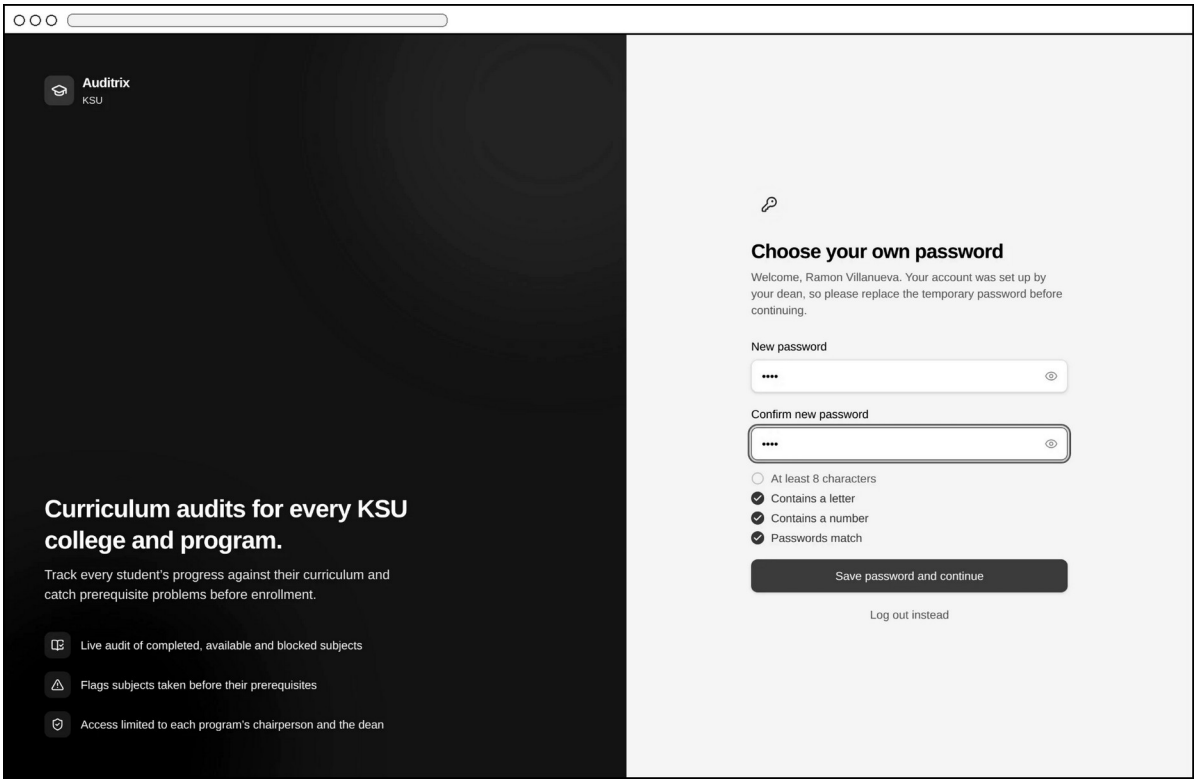


Figure 4.7l. First Log In — password change with the rules checklist

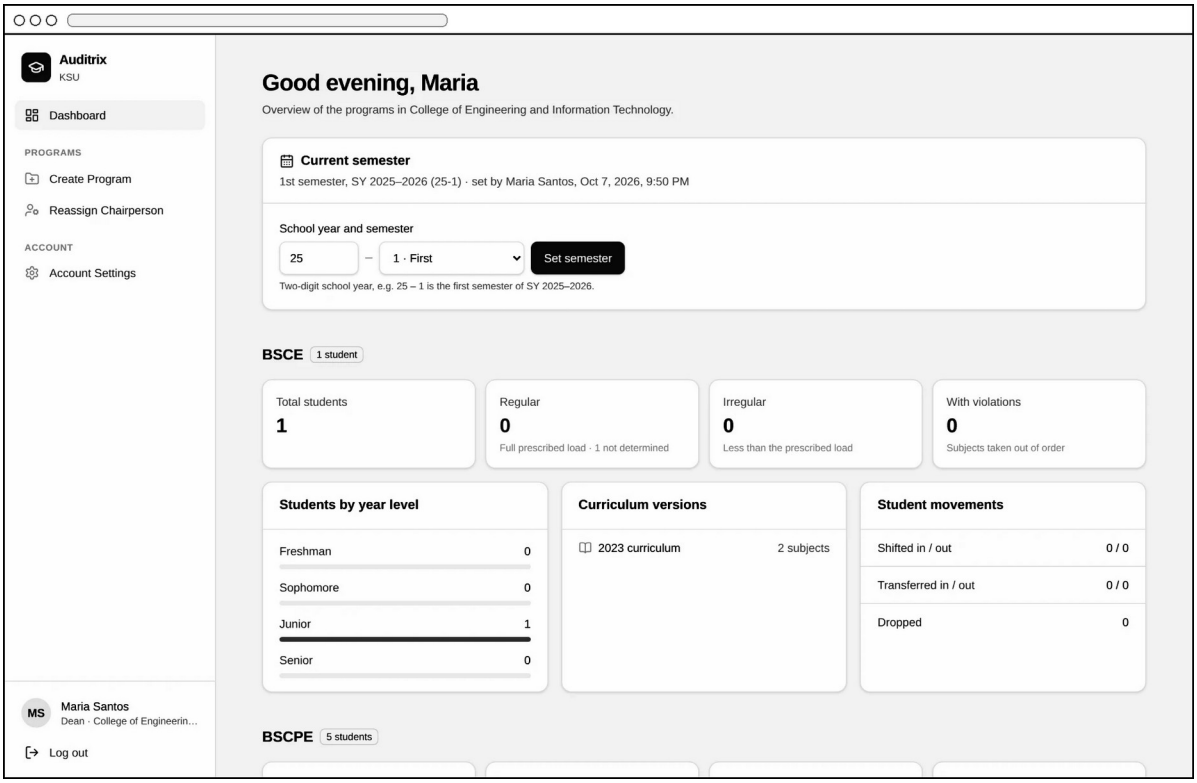


Figure 4.7m. Dean Dashboard – Current semester

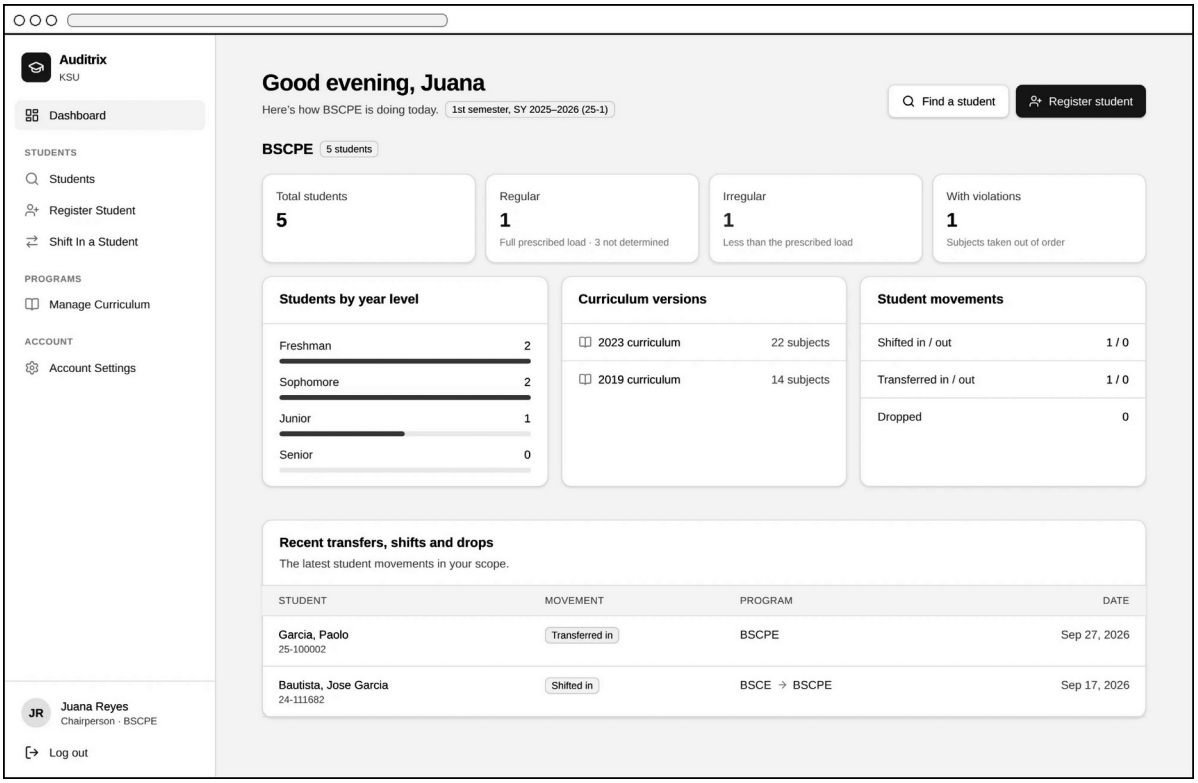


Figure 4.7n. Chairperson Dashboard

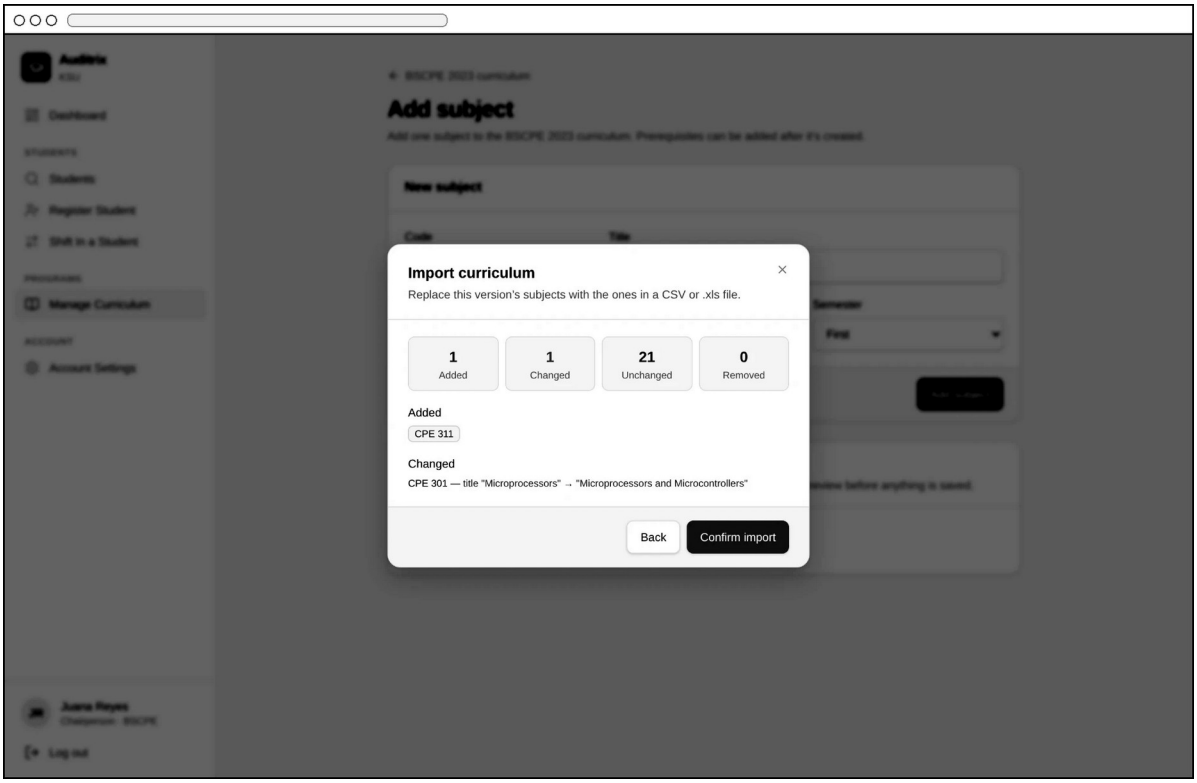


Figure 4.7o. Import Curriculum – preview of changes

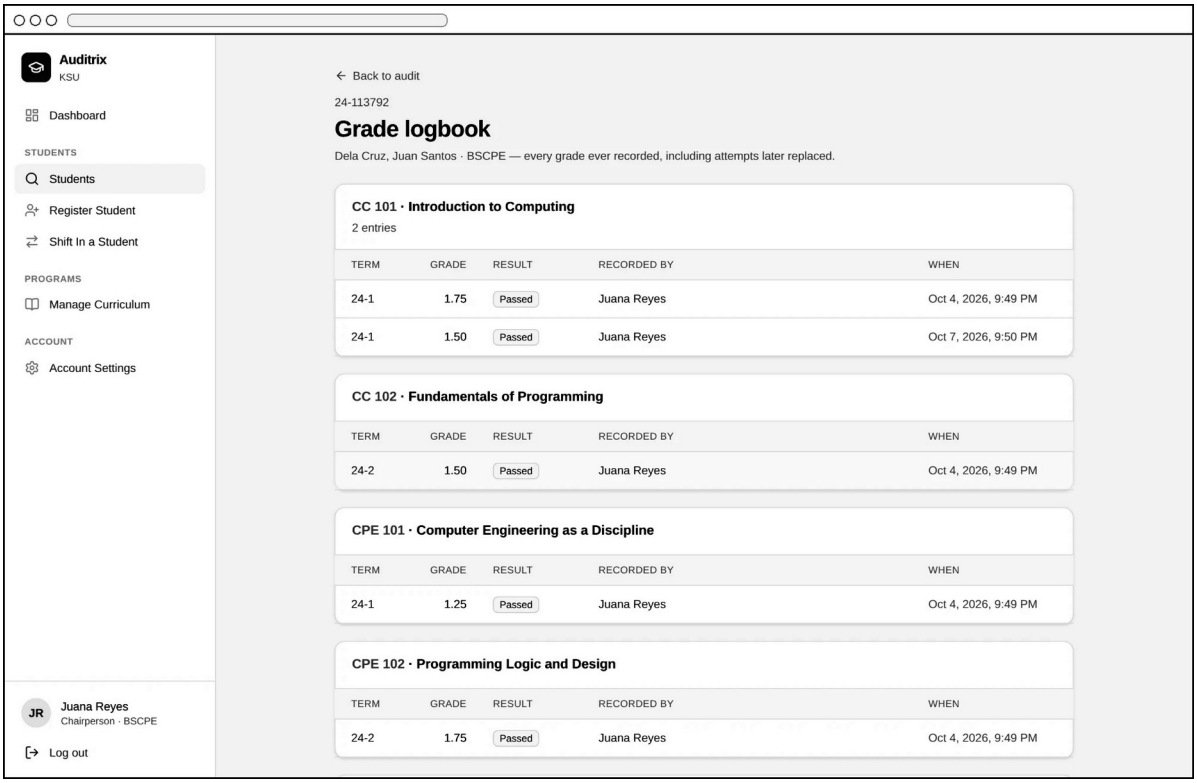


Figure 4.7p. Grade Logbook

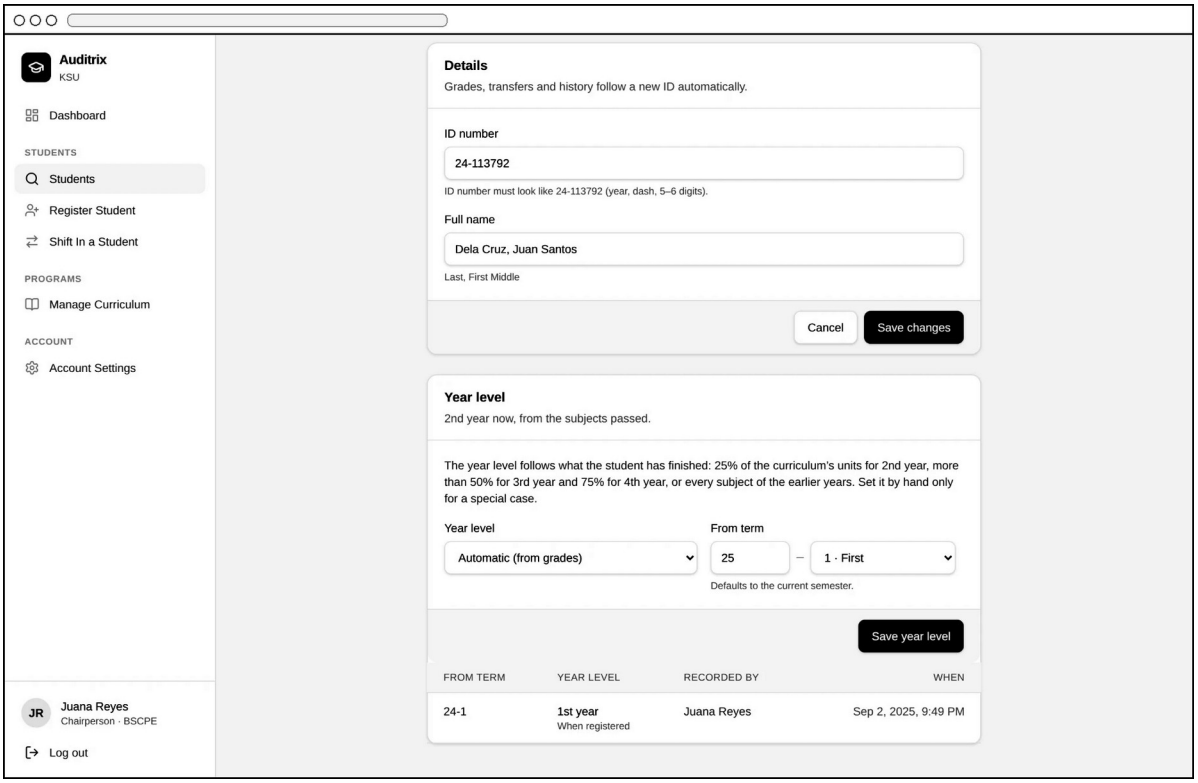


Figure 4.7q. Edit Student — year level (automatic or set by hand)

Non-Functional Requirements of the Proposed System

- **Functional Suitability:** the Audit Engine implements the prerequisite, co-requisite, year-standing and "all subjects completed" rules of the KSU Student Handbook and Operations Manual, verified by 163 automated tests.
- **Reliability:** audit results are worked out from the terms every time, so they do not depend on the order of grade entry; invalid grades, terms, ID numbers and import files are rejected with a reason, and curriculum imports

are previewed before saving; the database itself refuses grades that are not KSU grades and changes to locked records.

- **Performance Efficiency:** the audit is computed when a student's page or the dashboard is opened, so results are current as soon as grades are saved.
- **Usability and Security:** the interface uses one consistent design with confirmation dialogs and notifications; access is role-scoped via RLS, and new chairpersons must replace their temporary password at first login; every page checks the user's role, users are signed out after 30 minutes without activity, and an account is blocked for 15 minutes after five wrong passwords.
- **Maintainability:** curriculum versioning allows new curriculum years to be added without altering audit logic for existing cohorts; the audit rules are kept in a separate, tested module, and each database change has an undo script.
- **Portability/Compatibility:** delivered as a responsive web application accessible across desktop, tablet, and smartphone browsers.

Testing of the System

Two kinds of testing were carried out on the developed system: automated unit and integration testing, and functional (end-to-end) testing. User acceptance testing with actual chairpersons and deans, and testing on a hosted (online) deployment, have not yet been done.

Automated testing. A total of 163 automated tests were written using the test runner built into Node.js, and all 163 pass. The tests need no database or network: the server actions are tested with Supabase replaced by an in-memory fake. Table 4.3 summarizes what they check.

Area	What is checked
Audit engine	Prerequisite chains; pending prerequisites; prerequisites checked by term (same term, later term, never passed, out-of-order credit, retake clears it, midyear ordering, waiting for a grade); INC (resolved term, taken while INC); co-requisites (passed together, one failed, different terms, one without the other, waiting); "all subjects completed"
Year level and standing	Student Handbook and Operations Manual thresholds (25%, more than 50%, 75%); finishing earlier years; only grades before the current semester; failed and out-of-order subjects excluded; registered level and chairperson override; year standing as of the term taken; "please verify"
Regular / irregular	Full load; back subjects on top; already passed; missing subjects; failed earlier; graded in the current term; not determined (no enrollment, no semester set, nothing prescribed)
Grade entry	KSU grade values; rejected rows with reasons; same-term violations in any row order; entry order does not matter; violations survive re-saving; INC completion and its term; earlier-term protection and confirmed correction; malformed terms
Imports	Curriculum CSV / .xls parsing and the import preview plan; reading CSV, Excel workbooks (.xlsx), KSU-MIS .xls and Excel re-saved .xls; old Excel 97-2003 and damaged files; student template and .xlsx import
Other rules	Subject-code matching, student ID format, search terms, password policy and its checklist
Server actions and permissions	Registering, importing and editing students (ID validation); dean and chairperson permission checks for reassigning chairpersons, creating programs and accepting shift-ins

Table 4.3. Summary of the Automated Tests

Functional testing. The researchers scripted a web browser with Playwright against a local copy of the full system (the PostgreSQL database and the application) loaded with sample CEIT data, logging in as a dean and as chairpersons and going through each feature: violations and

their reasons, INC completion, the out-of-order warning after saving grades, setting the current semester, automatic year levels moving up with grades and semesters, chairperson overrides, regular and irregular counts, .xls and .xlsx imports, the earlier-term warning, and the password checklist. All scenarios passed after the defects found were fixed.

A problem found and fixed. The first version of the prerequisite checker decided, at the moment a grade was typed in, whether the prerequisites were already in the database, and stored the result as a permanent "invalid entry" flag. Testing showed that this gave wrong results in five situations:

- grades entered out of order (e.g., second-semester grades before first-semester grades) were wrongly flagged;
- a prerequisite passed only in a later term was not caught if it happened to be entered first;
- same-term prerequisite violations were caught or missed depending on the order of the rows;
- co-requisite results depended on the order of entry; and

- re-saving a flagged grade after the prerequisite was entered erased the violation.

The checker was therefore redesigned to work from the terms: it compares the term each subject was taken with the term each requirement was passed, every time the audit is shown. The result no longer depends on the order in which grades are entered, and each of the five situations above is now covered by an automated test. This experience shows why a curriculum audit should be based on when subjects were taken rather than on when their grades were encoded: in practice, the grades of existing students are often encoded long after the terms in which they were earned.

Extent of Compliance of the Proposed System (SOP No. 4)

Upon completion of the system, Auditrix is to be evaluated by the Program Chairpersons and selected IT experts using a printed (paper-based) questionnaire built on the ISO/IEC 25010 software quality model, with each characteristic mapped to a specific part of the system. Responses will be analyzed using the Weighted Mean. Table 4.4 will present the extent of compliance per quality characteristic.

Quality Characteristic	System Component Evaluated	Weighted Mean	Verbal Interpretation
Functional Suitability	Audit Engine / grade entry and violation flagging	[insert]	[insert]
Reliability	Consistent audit results / validated imports	[insert]	[insert]
Performance Efficiency	Audit and dashboard loading time	[insert]	[insert]
Usability	Dashboard and student audit page	[insert]	[insert]
Security	RLS access control / password policy	[insert]	[insert]
Compatibility	Cross-device responsiveness	[insert]	[insert]
Maintainability	Curriculum versioning / modular audit rules	[insert]	[insert]
Portability	Web-based accessibility	[insert]	[insert]

Quality Characteristic	System Component Evaluated	Weighted Mean	Verbal Interpretation
Safety	Data integrity / confirmations / access safeguards	[insert]	[insert]

Table 4.4. Extent of Compliance of Auditrix with ISO/IEC 25010